Report for AI Capstone Homework 2: 3D Scene Reconstruction

FU, YONG-WEI (112550107)

April 9, 2026

1 Implementation

1.1 Code: Pipeline Explanation

The 3D scene reconstruction pipeline was implemented using Python, NumPy, and Open3D. Below is a detailed breakdown of the techniques and algorithms used in each stage:

1. **Data Collection, Loading & Depth Quantization Fix:** *load.py* was first used to traverse the room and collect the rgb, depth, and semantic images. Later on, *reconstruct.py* was used to load the collected data and to reconstruct the 3D scene in point cloud format. The code was modified to preserve the depth information (maximum 10 meters) as a 16-bit unsigned integer, rather than as a 8-bit integer. This preserved accuracy and adhered to the specification's `DEPTH_SCALE = 1000.0`. The ground truth trajectory was also loaded into the process here.

2. **Unprojection:** Took the first image and used the depth information to unproject the rgb points into 3D space in point cloud format.

3. **Pre-processing:** Applied Voxel Downsampling to reduce the point count. Later, calculated the normal vectors of the points, so that the Fast Point Feature Histograms (FPFH) can extract local geometric features for each point, which will be later used to align the point clouds.

4. **Registration:**

   - **Global Registration (RANSAC):** After obtaining the FPFH features of two point clouds with the procedure above, RANSAC was used to roughly align the latter point cloud to the previous one. I utilized a high iteration count (up to 1.5M) to ensure a better probability of success.
   - **Local Registration (ICP):** Used Point-to-Plane ICP to refine the RANSAC result by minimizing the distance between corresponding points. Lastly, add them to the accumulated point cloud, store the new camera pose (by multiplying the previous pose with the estimated transformation) and repeat the process until the end of the collected data.

5. **Post-processing (Ceiling Removal):** Implemented a height-based mask to remove the ceiling for visualization. This is integrated into the evaluation phase rather than the reconstruction phase to maintain the geometric integrity of the raw data.

1.2 Result & Discussion

1.2.1 Screenshots

Figure 1: Floor 1 Reconstruction Overview: (Top) Bird's-eye views of apartment_0; (Middle) Side views; (Bottom) Close-up view.

1.2.2 Comparison Table

The following table summarizes the hyperparameter search conducted to optimize for both Mean L2 Error and execution time. (the experiment were run on the second floor.)

Exp #	Voxel (m)	RANSAC Iter	RANSAC Thresh / Voxel	FPFH max_nn	Total Points	Mean L2 (m)	Time (s)
0	0.04	1,500,000	2.5	100	196,933	**0.038497**	81.39
1	0.04	1,500,000	0.1	100	185,894	0.130952	176.67
2	0.08	1,500,000	0.2	100	54,812	0.201518	**53.00**
3	0.04	100,000	0.1	100	180,029	0.198384	80.77
4	0.04	1,500,000	0.048	100	177,681	0.130922	224.23
5	0.04	1,500,000	0.1	50	197,248	0.038534	188.64

Table 1: Hyperparameter tuning, trajectory error, and execution time.

1.2.3 Discussion & Findings

Based on the experimental results, several key insights were identified:

- **The Iteration Bottleneck (Exp 3):** Reducing RANSAC iterations from 1.5M to 100k caused the Mean L2 Error to jump significantly (from 0.038m to 0.198m). Because the agent's step size in Habitat is 0.25m, the inadequate steps would lead to a extremely low inlier ratio. Therefore, we can see that the stochastic nature of RANSAC requires a higher "search budget" to find the correct transformation; otherwise, it converges to local minima.

- **Voxel Trade-offs (Exp 2):** Increasing the voxel size to 0.08m achieved the fastest execution time (53s), but significant cost to accuracy was incurred. The coarse geometry makes it difficult for Point-to-Plane ICP to calculate accurate normals, leading to increased odometry drift.
- **RANSAC Thresholding (Exp 4):** This experiment demonstrated that a tight RANSAC threshold ($0.048 \times 0.04 = 0.00192 \simeq 2$ mm) makes the system less robust to Habitat's discrete movement. If the threshold is smaller than the noise or the actual jump between frames, RANSAC fails to identify valid correspondences, leading to higher error and awful trajectory estimation.
- **Feature Optimization (Exp 5):** Reducing the FPFH nearest neighbors to 50 maintained high accuracy (0.038m). This suggests that for indoor environments with many planar surfaces, local geometric signatures do not require high neighbor density to remain descriptive.

1.2.4 Custom ICP Implementation (Bonus)

As a bonus task, a custom Point-to-Plane ICP algorithm was implemented from scratch. The algorithm linearizes the rotation matrix using small-angle approximations and solves the resulting system of linear equations using the least-squares method.

Implementation	Mean L2 Error (m)	Exec Time (s)	Reliability
Open3D (`Point-to-Plane`)	0.038497	81.39	High
Custom (`Point-to-Plane`)	0.058512	93.25	A bit lower than Open3D

Table 2: Comparison between Open3D and Custom ICP implementations.

2 Questions

2.1 What happens if you perform ICP without Global Registration (RANSAC)? Why?

If standard ICP is performed without an initial coarse alignment from RANSAC, the algorithm will frequently fail or converge into a local minimum. I would say that ICP is "taking small but concise steps", where RANSAC is "taking big but noisy steps". It is merely a local optimization algorithm, therefore it's nearly impossible to achieve global convergence without a good initial guess. To obtain that initial guess, RANSAC is thus put into use before ICP.

2.2 Describe any tricks used to improve your ICP stability.

To meet the execution time constraints while maintaining accuracy, several optimization and stability techniques were implemented:

- **Stochastic Point-to-Plane Minimization:** To overcome the computational bottleneck of Python's `for`-loops, a random sampling ratio of 10% was applied to the source point cloud. By calculating the Hessian matrix ($A^T A$) using a statistically representative subset of points, the algorithm achieved a significant speedup without a proportional loss in registration accuracy.
- **Vectorized Normal Equations:** Instead of incrementally updating the system matrix inside the correspondence loop, the point-to-plane Jacobians were stacked into a matrix A and solved using vectorized NumPy operations ($A^T Ax = A^T b$). This shifted the heavy computation to optimized C++/BLAS backends, reducing per-iteration time by over 80%.
- **Adaptive Distance Rejector:** A squared-distance threshold of $0.0025m^2$ (corresponding to a $5cm$ search radius) was used to filter out outliers. This ensures that only points with a valid physical neighbor contribute to the pose update, preventing "phantom" geometry from causing odometry drift.
- **Convergence-Based Early Exit:** The algorithm monitors the magnitude of the 6D transformation vector. If the L2-norm of the update drops below $1e^{-3}$, the loop terminates early. This prevents the system from wasting cycles on sub-millimeter "jitter" that does not contribute to the final map's visual quality.